

ОДЕСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ІМЕНІ І. І. МЕЧНИКОВА
ФАКУЛЬТЕТ МАТЕМАТИКИ, ФІЗИКИ І ІНФОРМАЦІЙНИХ ТЕХНОЛОГІЙ
КАФЕДРА ОПТИМАЛЬНОГО КЕРУВАННЯ І ЕКОНОМІЧНОЇ КІБЕРНЕТИКИ

Паралельні алгоритми обчислювальної математики

Звіт

з лабораторної роботи №4

Паралельне сортування парно-непарними перестановками з MPI

Виконав: студент 1 курсу магістратури
спеціальності 113 Прикладна математика
Чачко Натан Леонідович

2025 р.

Послідовна програма сортування парно-непарними перестановками.

```

1 void Odd_even_sort(int a[] /* in/out */, int n /* in */) {
2     int phase, i, temp;
3     for (phase = 0; phase < n; phase++)
4         if (phase % 2 == 0) { /* Even phase */
5             for (i = 1; i < n; i += 2)
6                 if (a[i-1] > a[i]) {
7                     temp = a[i];
8                     a[i] = a[i-1];
9                     a[i-1] = temp;
10                }
11        } else { /* Odd phase */
12            for (i = 1; i < n-1; i += 2)
13                if (a[i] > a[i+1]) {
14                    temp = a[i];
15                    a[i] = a[i+1];
16                    a[i+1] = temp;
17                }
18        }
19 } /* Odd even sort /

```

Розглянемо тепер паралельний алгоритм, в якому кожен процес зберігає $n/p > 1$ елементів. (Ми вважаємо, що n без залишку ділиться на p .)

Фактично приклад з Таблиці 4 ілюструє найгіршу продуктивність цього алгоритму:

Теорема 1 *Якщо паралельне сортування непарно-парне перестановками виконується p процесами, то після p фаз масив буде відсортований.*

Паралельний алгоритм можна описати так:

```

1 Sort_local_keys;
2 for (phase = 0; phase < comm_sz; phase++) {
3     partner = Compute_partner(phase, myrank);
4     if (I'm not idle) {
5         Send_my_keys_to_partner;

```

Time	0	1	2	3
Start	15, 11, 9, 16	3, 14, 8, 7	4, 6, 12, 10	5, 2, 13, 1
After Local Sort	9, 11, 15, 16	3, 7, 8, 14	4, 6, 10, 12	1, 2, 5, 13
After Phase 0	3, 7, 8, 9	11, 14, 15, 16	1, 2, 4, 5	6, 10, 12, 13
After Phase 1	3, 7, 8, 9	1, 2, 4, 5	11, 14, 15, 16	6, 10, 12, 13
After Phase 2	1, 2, 3, 4	5, 7, 8, 9	6, 10, 11, 12	13, 14, 15, 16
After Phase 3	1, 2, 3, 4	5, 6, 7, 8	9, 10, 11, 12	13, 14, 15, 16

Табл. 4: Паралельне сортування парно-непарними перестановками

```

1     Receive_keys_from_partner;
2     if (myrank < partner)
3         Keep_smaller_keys;
4     else
5         Keep_larger_keys;
6     }
7 }

```

Обчислюєм ранг партнера

```

1 if (phase % 2 == 0) /* Even phase */
2     if (myrank % 2 != 0) /* Odd rank */
3         partner = myrank - 1;

```

```

4     else /* Even rank */
5         partner = myrank + 1;
6 else /* Odd phase */
7     if (my rank % 2 != 0) /* Odd rank */
8         partner = myrank + 1;
9     else /* Even rank */
10        partner = myrank - 1;
11 if (partner == -1 || partner == comm_sz)
12 partner = MPI_PROC_NULL;

```

MPI_PROC_NULL - константа, що визначається MPI.

Є сенс реалізувати надсилання та отримання повідомлень за допомогою одного виклику MPI_Sendrecv:

```

1 MPI_Sendrecv(my_keys, n/comm_sz, MPI_INT, partner, 0,
2   recv_keys, n/comm_sz, MPI_INT, partner, 0, comm,
3   MPI_Status ignore);

```

Залишилося визначити, які ключі слід зберегти. Щоб знизити витрати, враховуючи той факт, що ми вже маємо два відсортованих списки з n/p ключів, об'єднаємо два списки в один. Більш того, нам не потрібне повне злиття двох списків: достатньо знайти, наприклад, найменші n/p ключі.

```

1 void Merge_low( int my_keys[], /* in/out */
2   int recv_keys[], /* in */
3   int temp_keys[], /* scratch */
4   int local_n /* = n/p, in */) {
5   int m_i, r_i, t_i; m_i = r_i = t_i = 0;
6   while (t_i < local_n) {
7       if (my_keys[m_i] <= recv_keys[r_i]) {
8           temp_keys[t_i] = my_keys[m_i];
9           t_i++; m_i++;
10      } else {
11          temp_keys[t_i] = recv_keys[r_i];
12          t_i++; r_i++;
13      }
14  }
15  for (m_i = 0; m_i < local_n; m_i++)
16      my_keys[m_i] = temp_keys[m_i];
17 } /* Merge_low */

```

Провести обчислювальний експеримент та заповнити таблицю 5.

Побудувати графіки залежності часу сортування від кількості процесів для різних n .

До звіту включити відомості про обчислювальну систему та реалізацію MPI, що використовувались для обчислень.

Повний код програми

Було додано створення масиву випадкових чисел довжини n кожним процесом окремо, зчитування n з аргументів команди термінала, вимірювання часу виконання програми.

```
1 #include <stdio.h>
2 #include <mpi.h>
3 #include <stdlib.h>
4 #include <tgmath.h>
5 #include <time.h>
6
7
8 int *my_keys;
9
10 void Bubble_sort(int a[] /* in/out */,
11                 int n /* in */) {
12     int list_length, i, temp;
13     for (list_length = n; list_length >= 2; list_length--)
14         for (i = 0; i < list_length - 1; i++)
15             if (a[i] > a[i + 1]) {
16                 temp = a[i];
17                 a[i] = a[i + 1];
18                 a[i + 1] = temp;
19             }
20 }
21
22 void Merge_smaller(
23     int my_keys[], /* in/out */
24     int recv_keys[], /* in */
25     int temp_keys[], /* scratch */
26     int local_n /* = n/p, in */) {
27     int m_i, r_i, t_i;
28     m_i = r_i = t_i = 0;
29     while (t_i < local_n) {
30         if (my_keys[m_i] <= recv_keys[r_i]) {
31             temp_keys[t_i] = my_keys[m_i];
32             t_i++;
33             m_i++;
34         } else {
35             temp_keys[t_i] = recv_keys[r_i];
36             t_i++;
37             r_i++;
38         }
39     }
40     for (m_i = 0; m_i < local_n; m_i++)
41         my_keys[m_i] = temp_keys[m_i];
42 }
43
44
45 void Merge_larger(
46     int my_keys[], /* in/out */
47     int recv_keys[], /* in */
48     int temp_keys[], /* scratch */
49     int local_n /* = n/p, in */) {
50     int m_i, r_i, t_i;
51     m_i = r_i = t_i = local_n - 1;
52     while (t_i >= 0) {
53         if (my_keys[m_i] >= recv_keys[r_i]) {
54             temp_keys[t_i] = my_keys[m_i];
55             t_i--;
56             m_i--;
```

```

57         } else {
58             temp_keys[t_i] = recv_keys[r_i];
59             t_i--;
60             r_i--;
61         }
62     }
63     for (m_i = 0; m_i < local_n; m_i++)
64         my_keys[m_i] = temp_keys[m_i];
65 }
66
67 int Compute_partner(int phase, int myrank, int comm_sz) {
68     int partner;
69     if (phase % 2 == 0) /* Even phase */
70         if (myrank % 2 != 0) /* Odd rank */
71             partner = myrank - 1;
72         else /* Even rank */
73             partner = myrank + 1;
74     else /* Odd phase */
75         if (myrank % 2 != 0) /* Odd rank */
76             partner = myrank + 1;
77         else /* Even rank */
78             partner = myrank - 1;
79     if (partner == -1 || partner == comm_sz)
80         partner = MPI_PROC_NULL;
81     return partner;
82 }
83
84
85 int main(int argc, char **argv) {
86     int size, ns;
87     int my_rank, comm_sz;
88     int phase, partner;
89     MPI_Status status;
90
91
92     int power = atoi(argv[1]);
93     long long n = pow(10, power);
94     int *a = malloc(sizeof(int) * n);
95
96
97     MPI_Init(&argc, &argv);
98     MPI_Comm_rank(MPI_COMM_WORLD, &my_rank);
99     MPI_Comm_size(MPI_COMM_WORLD, &comm_sz);
100
101     // Заповнення масиву випадковими значеннями
102     srand(time(nullptr) + my_rank);
103     for (long long i = 0; i < n; i++) {
104         a[i] = rand();
105     }
106     printf("\n=== n = 10~%d ===\n", power);
107     double start = MPI_Wtime();
108     size = n / comm_sz;
109     ns = size * my_rank;
110     int *recv_keys = malloc(size * sizeof(int));
111     int *temp_keys = malloc(size * sizeof(int));
112     my_keys = &a[ns];
113
114     // Sort_local_keys;
115     Bubble_sort(my_keys, size);
116
117     for (phase = 0; phase < comm_sz; phase++) {

```

```

118     partner = Compute_partner(phase, my_rank, comm_sz);
119     if (partner != MPI_PROC_NULL) {
120         // Send my keys to partner; Receive keys from partner;
121         MPI_Sendrecv(my_keys, size, MPI_INT, partner, 0,
122                     recv_keys, size, MPI_INT, partner, 0, MPI_COMM_WORLD,
123                     &status);
124         if (my_rank < partner) // Keep_smaller_keys;
125             Merge_smaller(my_keys, recv_keys, temp_keys, size);
126         else // Keep_larger_keys;
127             Merge_larger(my_keys, recv_keys, temp_keys, size);
128     }
129 }
130 printf("my_rank = %d; my_size = %d\n", my_rank, size);
131 const double end = MPI_Wtime();
132 if (my_rank == 0) {
133     printf("Elapsed time: %f seconds\n", end - start);
134 }
135
136 MPI_Finalize();
137 return 0;
138 }

```

Вивід програми:

```
C:\Users\natan.chachko\Desktop\UNI\5.2\Par_alg>mpiexec -n 2 .\cmake-build-release\p4 4
```

```

=== n = 10^4 ===
my_rank = 0; my_size = 5000
Elapsed time: 0.035929 seconds

```

```

=== n = 10^4 ===
my_rank = 1; my_size = 5000

```

```
C:\Users\natan.chachko\Desktop\UNI\5.2\Par_alg>mpiexec -n 4 .\cmake-build-release\p4 4
```

```

=== n = 10^4 ===
my_rank = 0; my_size = 2500
Elapsed time: 0.011816 seconds

```

```

=== n = 10^4 ===
my_rank = 1; my_size = 2500

```

```

=== n = 10^4 ===
my_rank = 2; my_size = 2500

```

```

=== n = 10^4 ===
my_rank = 3; my_size = 2500

```

```
C:\Users\natan.chachko\Desktop\UNI\5.2\Par_alg>mpiexec -n 8 .\cmake-build-release\p4 4
```

```

=== n = 10^4 ===
my_rank = 0; my_size = 1250
Elapsed time: 0.007540 seconds

```

```

=== n = 10^4 ===
my_rank = 7; my_size = 1250

```

```

=== n = 10^4 ===
my_rank = 4; my_size = 1250

```

```
=== n = 10^4 ===  
my_rank = 1; my_size = 1250
```

```
=== n = 10^4 ===  
my_rank = 6; my_size = 1250
```

```
=== n = 10^4 ===  
my_rank = 3; my_size = 1250
```

```
=== n = 10^4 ===  
my_rank = 5; my_size = 1250
```

```
=== n = 10^4 ===  
my_rank = 2; my_size = 1250
```

```
C:\Users\natan.chachko\Desktop\UNI\5.2\Par_alg>mpiexec -n 2 .\cmake-build-release\p4 5
```

```
=== n = 10^5 ===  
my_rank = 0; my_size = 50000  
Elapsed time: 4.980436 seconds
```

```
=== n = 10^5 ===  
my_rank = 1; my_size = 50000
```

```
C:\Users\natan.chachko\Desktop\UNI\5.2\Par_alg>mpiexec -n 4 .\cmake-build-release\p4 5
```

```
=== n = 10^5 ===  
my_rank = 3; my_size = 25000
```

```
=== n = 10^5 ===  
my_rank = 2; my_size = 25000
```

```
=== n = 10^5 ===  
my_rank = 0; my_size = 25000  
Elapsed time: 1.354272 seconds
```

```
=== n = 10^5 ===  
my_rank = 1; my_size = 25000
```

```
C:\Users\natan.chachko\Desktop\UNI\5.2\Par_alg>mpiexec -n 8 .\cmake-build-release\p4 5
```

```
=== n = 10^5 ===  
my_rank = 6; my_size = 12500
```

```
=== n = 10^5 ===  
my_rank = 4; my_size = 12500
```

```
=== n = 10^5 ===  
my_rank = 7; my_size = 12500
```

```
=== n = 10^5 ===  
my_rank = 0; my_size = 12500  
Elapsed time: 0.408089 seconds
```

```
=== n = 10^5 ===  
my_rank = 5; my_size = 12500
```

```
=== n = 10^5 ===  
my_rank = 3; my_size = 12500
```

```

=== n = 10^5 ===
my_rank = 2; my_size = 12500

=== n = 10^5 ===
my_rank = 1; my_size = 12500

C:\Users\natan.chachko\Desktop\UNI\5.2\Par_alg>mpiexec -n 2 .\cmake-build-release\p4 6

=== n = 10^6 ===
my_rank = 0; my_size = 500000
Elapsed time: 651.877869 seconds

=== n = 10^6 ===
my_rank = 1; my_size = 500000

C:\Users\natan.chachko\Desktop\UNI\5.2\Par_alg>mpiexec -n 4 .\cmake-build-release\p4 6

=== n = 10^6 ===
my_rank = 0; my_size = 250000
Elapsed time: 181.961606 seconds

=== n = 10^6 ===
my_rank = 2; my_size = 250000

=== n = 10^6 ===
my_rank = 3; my_size = 250000

=== n = 10^6 ===
my_rank = 1; my_size = 250000

C:\Users\natan.chachko\Desktop\UNI\5.2\Par_alg>mpiexec -n 8 .\cmake-build-release\p4 6

=== n = 10^6 ===
my_rank = 5; my_size = 125000

=== n = 10^6 ===
my_rank = 3; my_size = 125000

=== n = 10^6 ===
my_rank = 0; my_size = 125000
Elapsed time: 56.975520 seconds

=== n = 10^6 ===
my_rank = 4; my_size = 125000

=== n = 10^6 ===
my_rank = 6; my_size = 125000

=== n = 10^6 ===
my_rank = 7; my_size = 125000

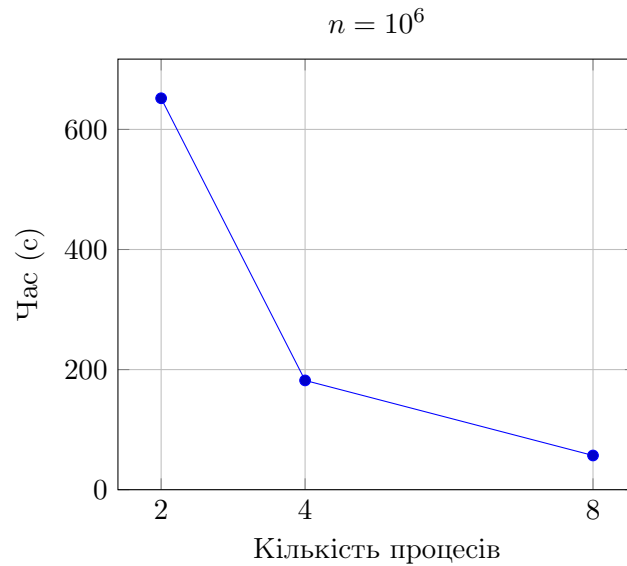
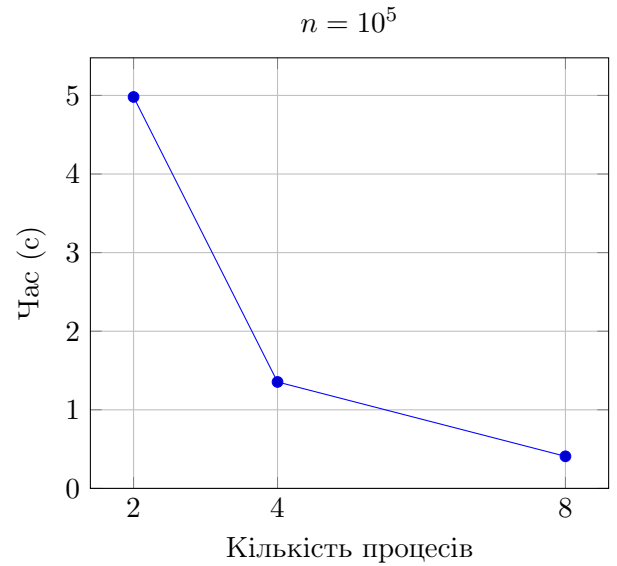
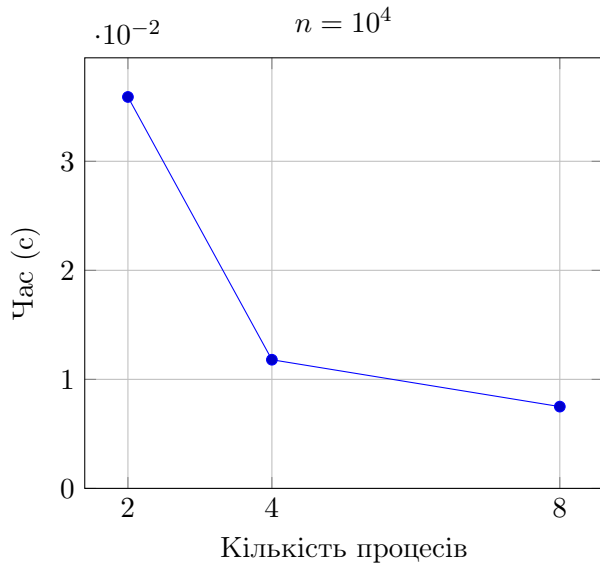
=== n = 10^6 ===
my_rank = 1; my_size = 125000

=== n = 10^6 ===
my_rank = 2; my_size = 125000

```


n	2	4	8
10^4	0.0359 s	0.0118 s	0.0075 s
10^5	4.9804 s	1.3543 s	0.4081 s
10^6	651.8779 s	181.9616 s	56.9755 s

Табл. 5: Час сортування масиву цілих чисел розміру n паралельною MPI-програмою з різною кількістю процесів.



Обчислення проводилися з використанням компілятора GCC (MinGW) на комп'ютері з операційною системою Windows 10 та наступними характеристиками:

CPU:
 11th Gen Intel(R) Core(TM) i7-1185G7 @ 3.00GHz 1.80 GHz
 Base speed: 1,80 GHz
 Cores: 4
 Logical processors: 8